Automated SSH Brute Force Detection & IP Blocking System Using Python + UFW + Postfix (mailutils)

Objective

To design and implement an automated security system that:

- Detects SSH brute-force attacks from system logs
- Blocks attacker IPs dynamically using firewall rules
- Sends real-time alerts using a local mail server
- Automatically unblocks IPs after a defined duration

Scope

This lab simulates a real-world SOC use case where:

- Continuous monitoring of authentication logs is required
- Automated incident response reduces manual effort
- Alerting ensures visibility of security events

Requirements

System Requirements

- Ubuntu Server
- Kali Linux

Software Components

- OpenSSH Server
- UFW Firewall
- Python 3
- Postfix Mail Server
- mailutils

Lab Architecture

Attacker Machine → SSH Login Attempts → /var/log/auth.log

↓

Python Script

↓

Detect Failed Login Attempts

↓

Block IP (UFW Firewall)

↓

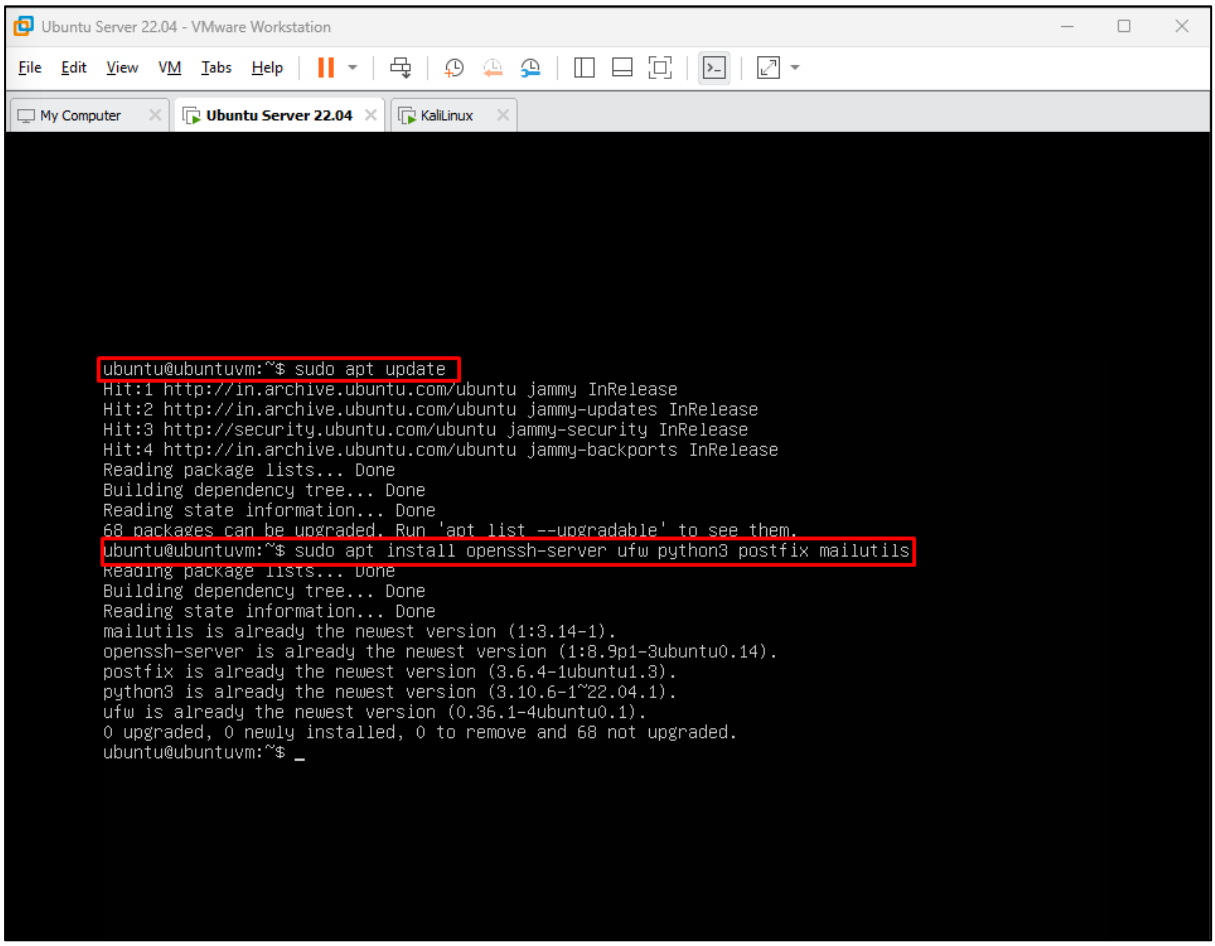
Send Alert (Postfix mailutils)

Implementation Procedure

Step 1: Install Required Packages

```
sudo apt update
```

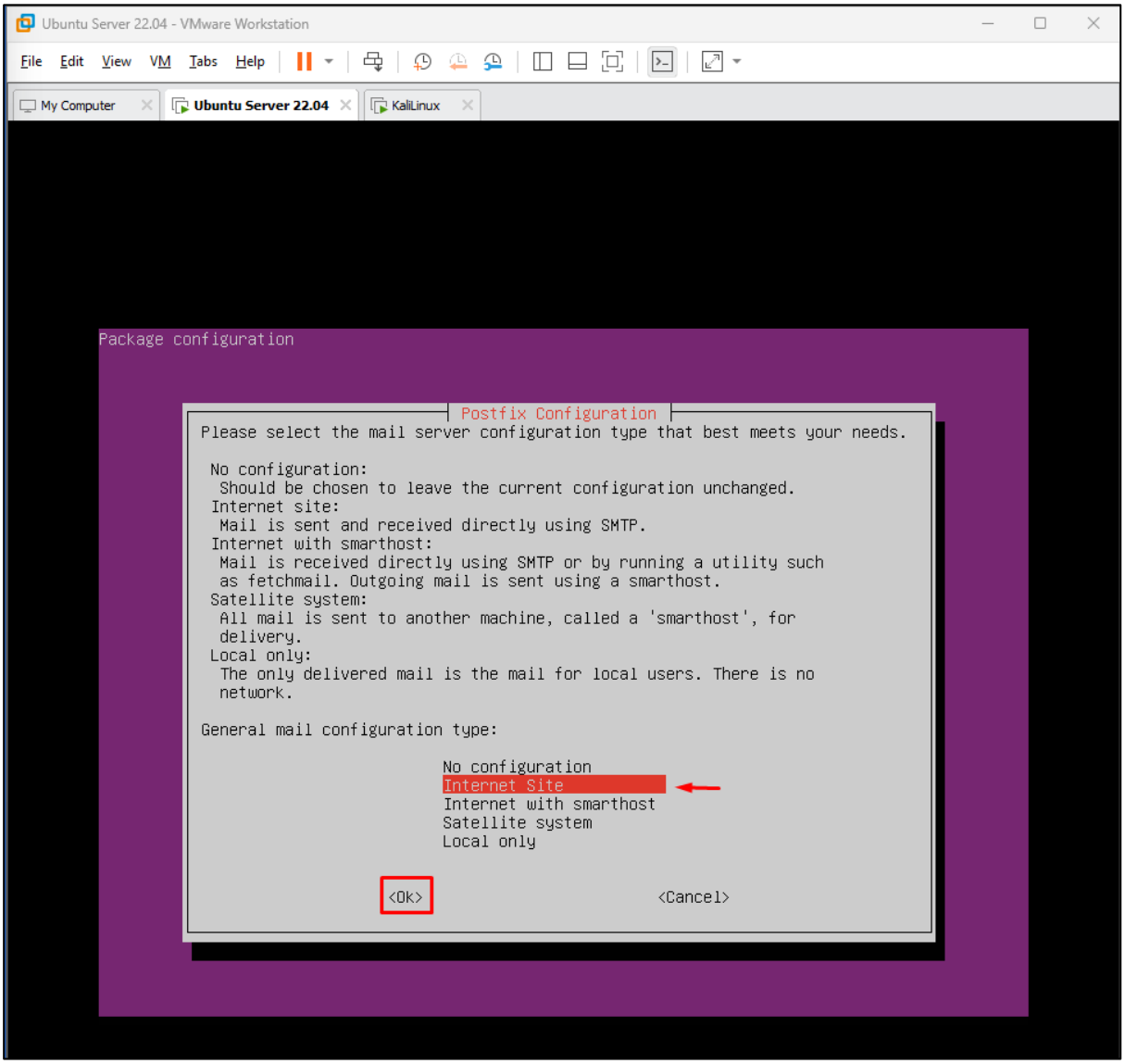
```
sudo apt install openssh-server ufw python3 postfix mailutils -y
```



Step 2: Configure Postfix

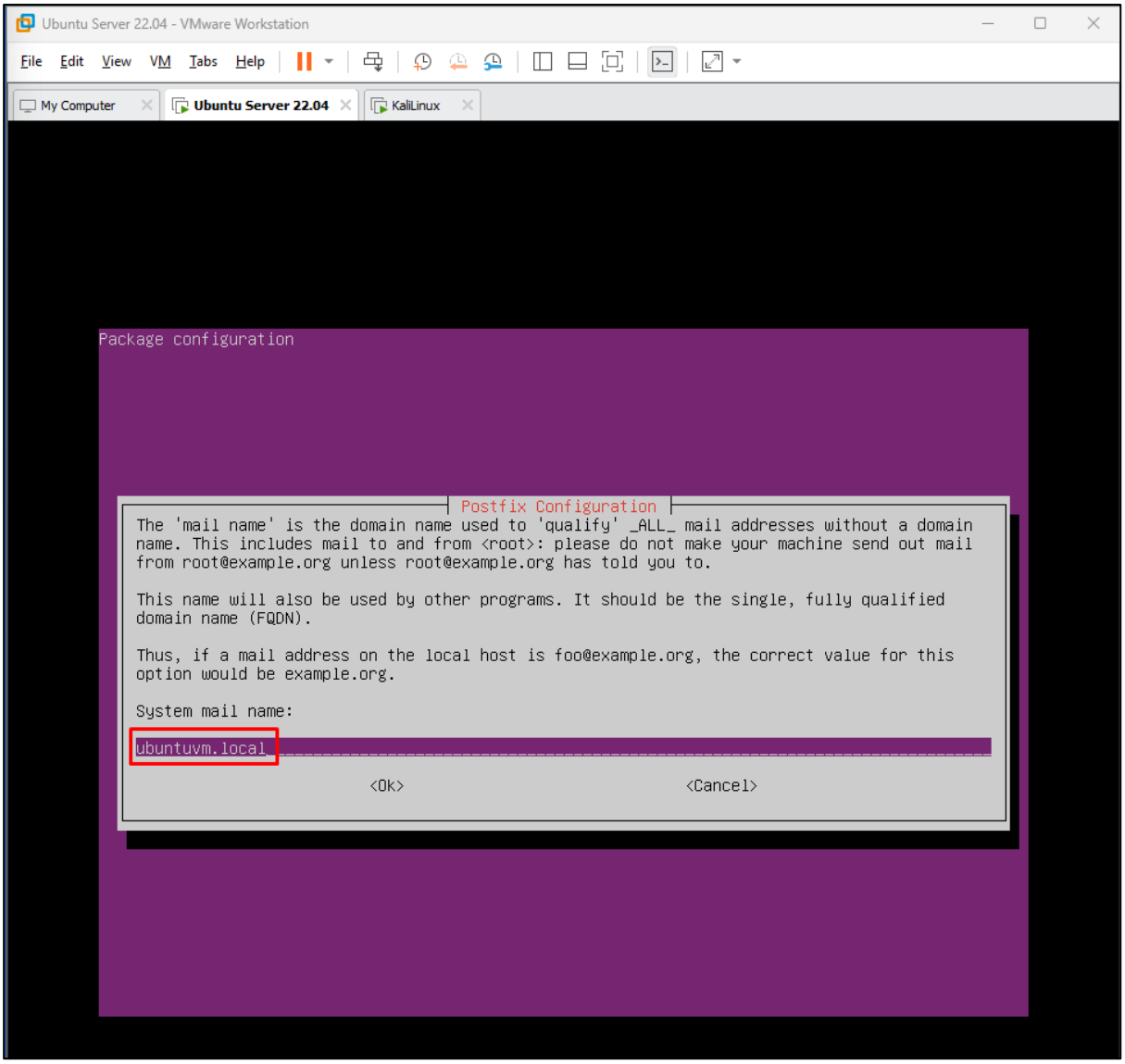
During installation:

- Select: **Internet Site**



- System mail name:

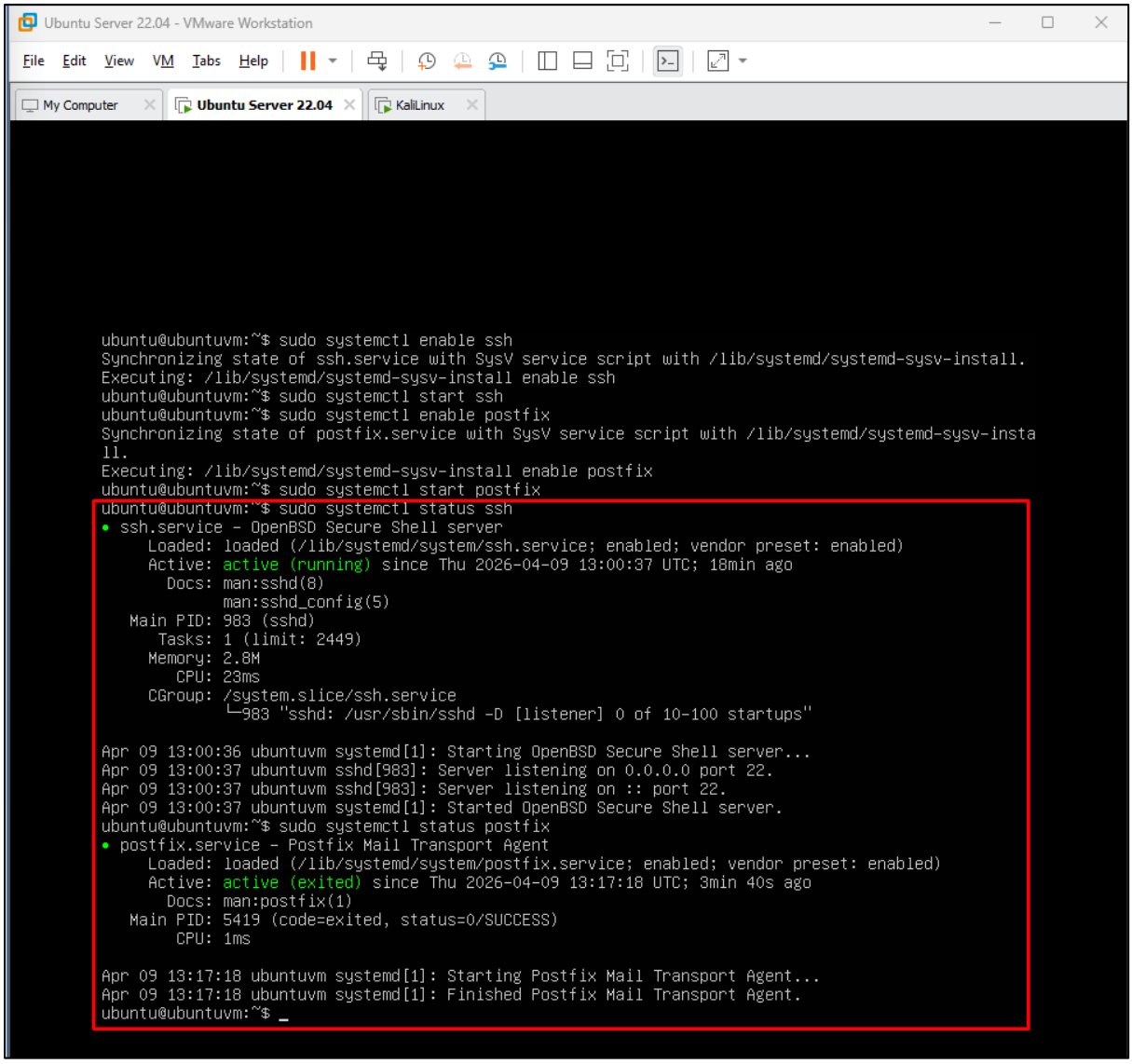
  ubuntuvm.local



Step 3: Start and Enable Services

```
sudo systemctl enable ssh
```

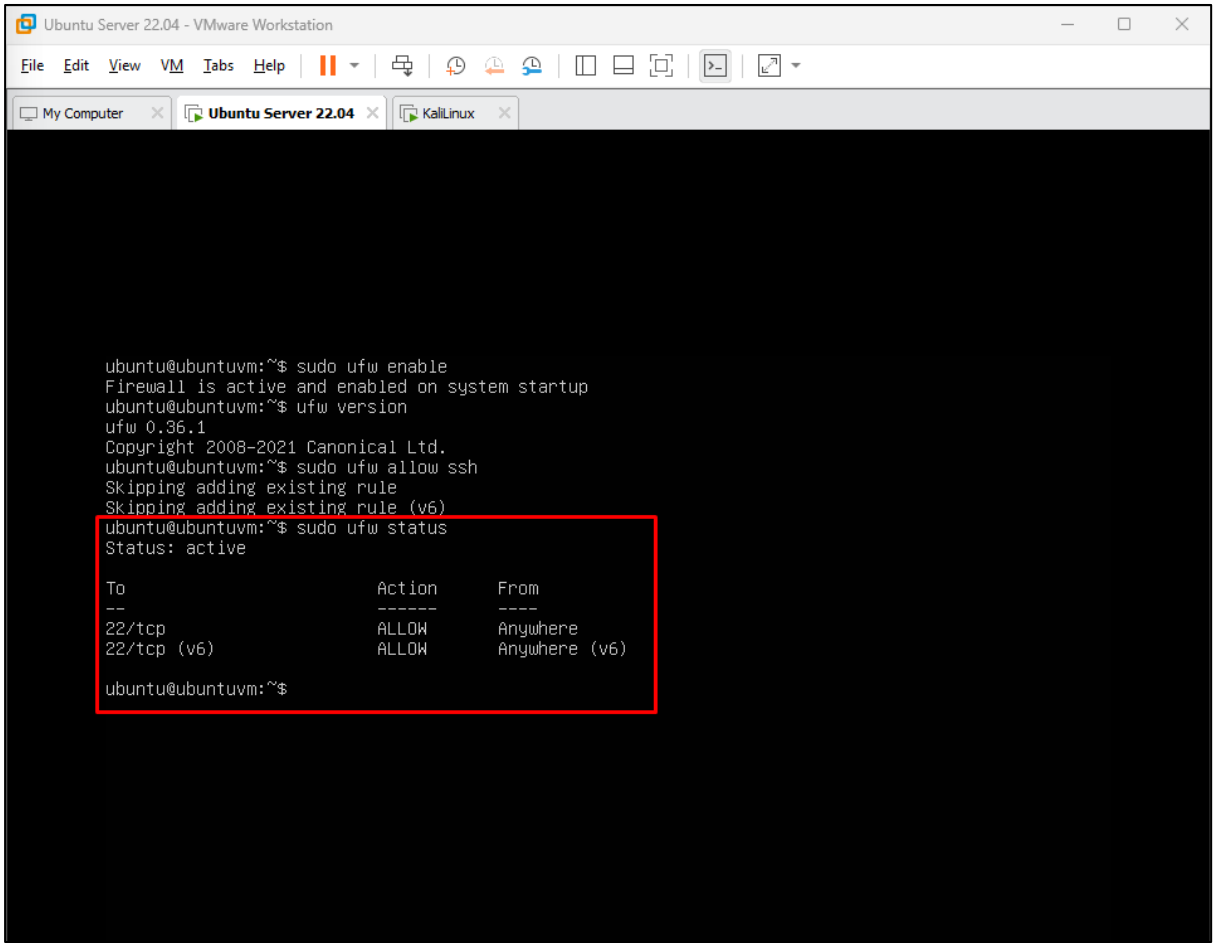
```
sudo systemctl start ssh

sudo systemctl enable postfix

sudo systemctl start postfix
```



Step 4: Configure Firewall

```
sudo ufw enable
```

```
sudo ufw allow ssh

sudo ufw status
```



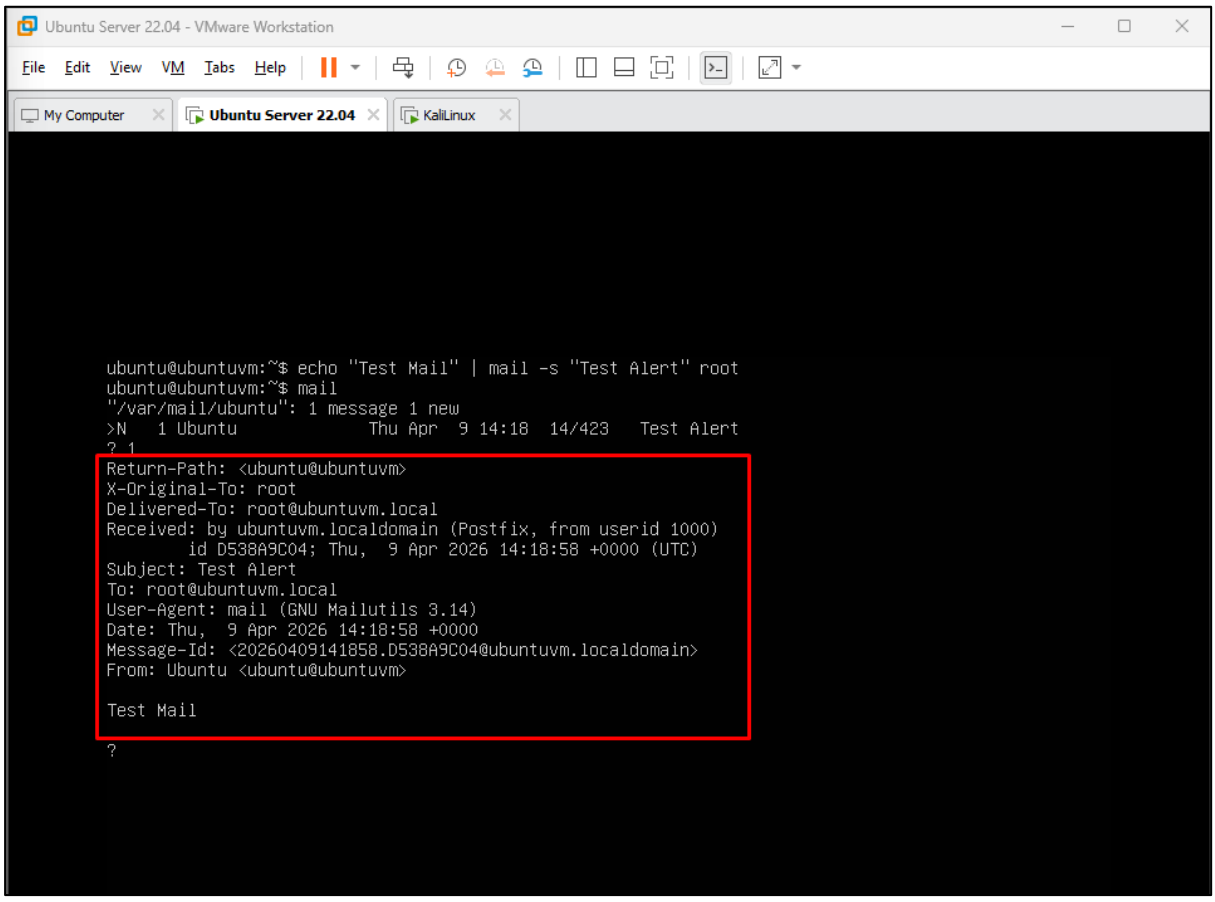
Step 5: Verify Mail Functionality

Send test mail:

```
echo "Test Mail" | mail -s "Test Alert" root
```

Check mailbox:

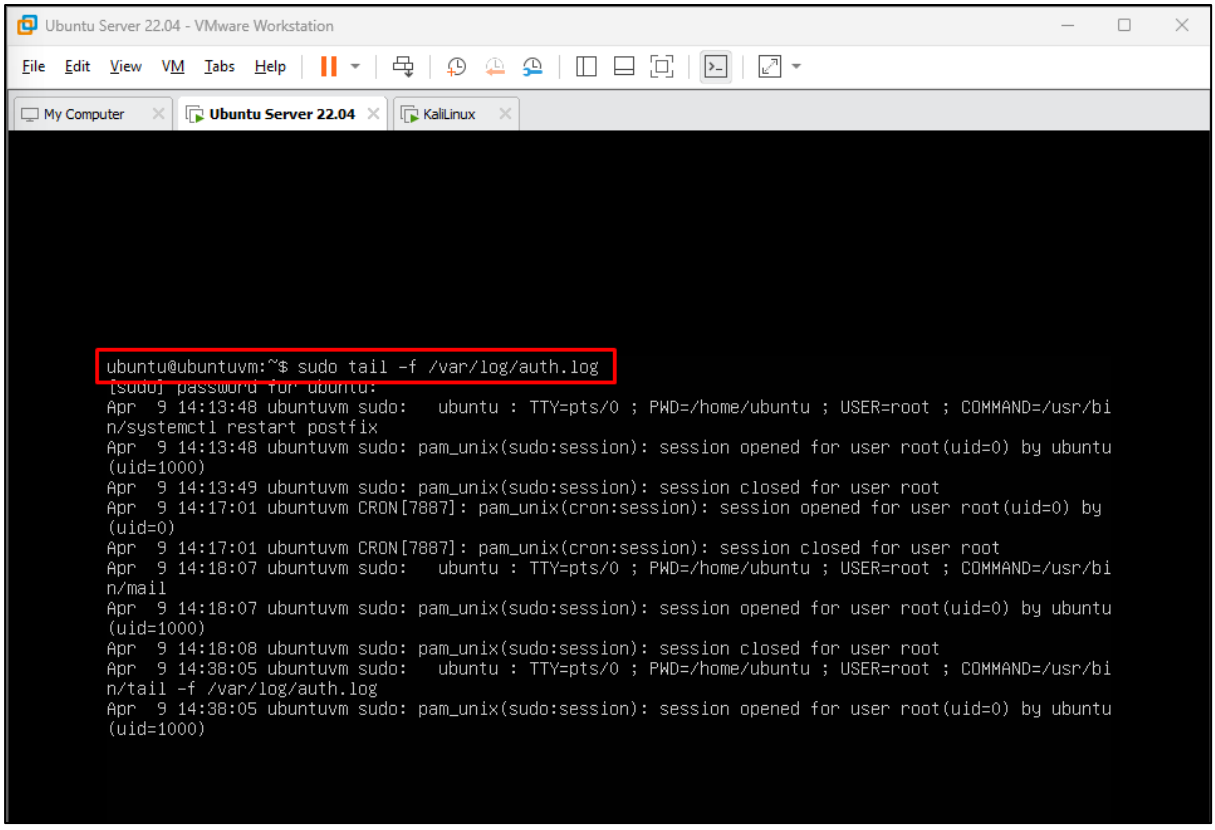
```
mail
```



Step 6: Verify SSH Logs

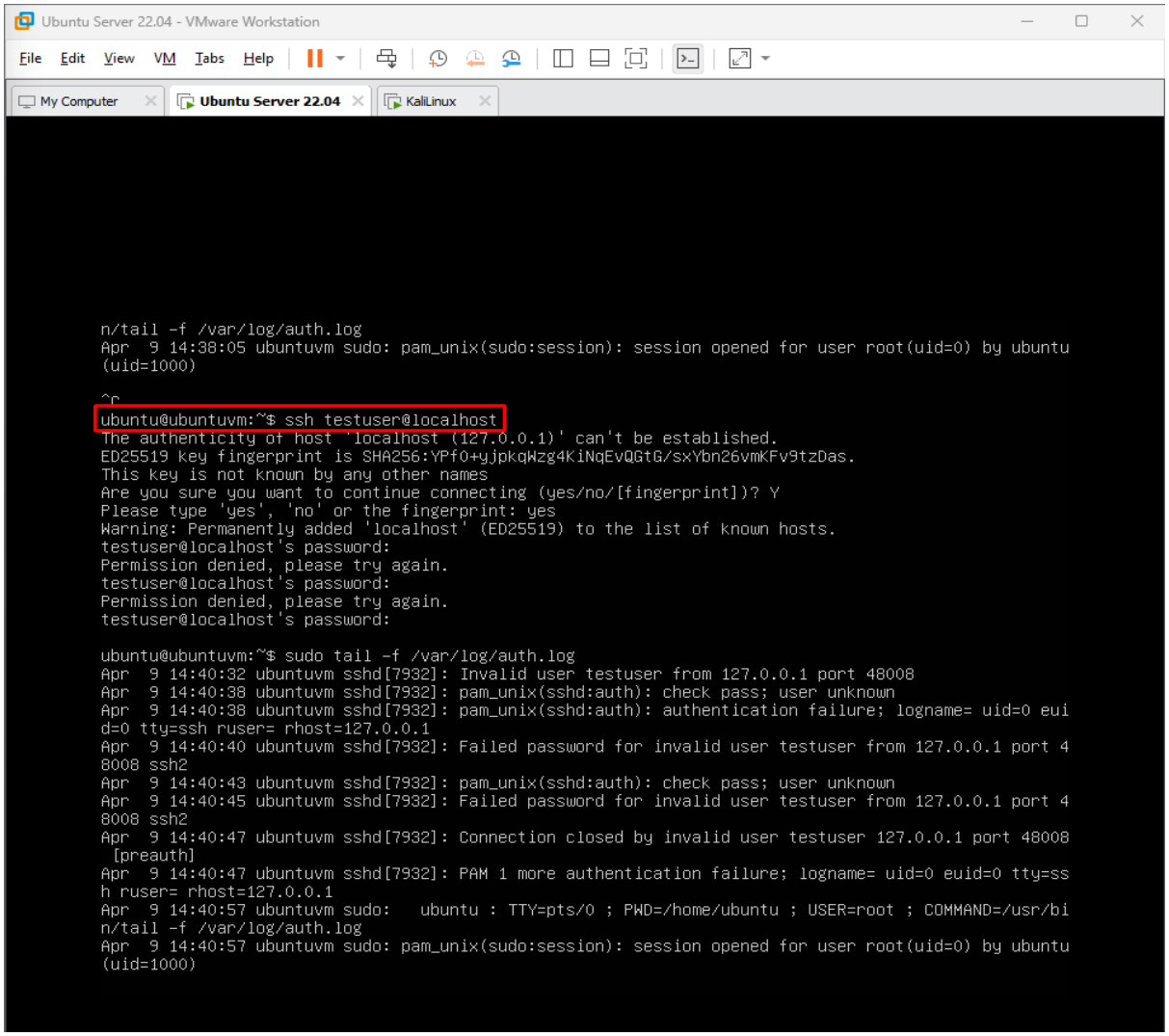
Monitor logs:

```
sudo tail -f /var/log/auth.log
```



Trigger test failure:

```
ssh testuser@localhost
```



Step 7: Create Detection Script

Create file:

```
nano ssh_monitor.py
```

Paste the script

```
import time

import re

import subprocess

import os

from collections import defaultdict


# ================= CONFIGURATION =================

LOG_FILE = "/var/log/auth.log"

THRESHOLD = 5            # Number of failed attempts before blocking
```

```
BLOCK_DURATION = 300     # Time in seconds to stay blocked (5 minutes)
ALERT_EMAIL = "root"     # Local user to receive Postfix alerts
# ==================================================

# Global dictionaries to track state
failed_attempts = defaultdict(int)
blocked_ips = {}

def send_alert(ip):
    """Sends an email alert using the local Postfix/mailutils setup."""
    subject = "SECURITY ALERT: SSH Brute Force Detected"
    body = f"The IP address {ip} has been blocked after {THRESHOLD} failed
login attempts."
    try:
        # Use mailutils to send the alert
        os.system(f'echo "{body}" | mail -s "{subject}" {ALERT_EMAIL}')
        print(f"[MAIL] Alert sent to {ALERT_EMAIL}")
    except Exception as e:
        print(f"[ERROR] Failed to send email: {e}")

def block_ip(ip):
    """Adds the IP to UFW deny list and records the timestamp."""
    print(f"[!] THRESHOLD REACHED: Blocking {ip}...")
    try:
        subprocess.run(["sudo", "ufw", "deny", "from", ip], check=True,
capture_output=True)
        blocked_ips[ip] = time.time()
        send_alert(ip)
    except subprocess.CalledProcessError as e:
        print(f"[ERROR] UFW block failed for {ip}: {e}")

def unblock_ips():
    """Checks the list of blocked IPs and removes them if time has
expired."""
    current_time = time.time()
    # Use list() to avoid 'dictionary changed size during iteration' error
```

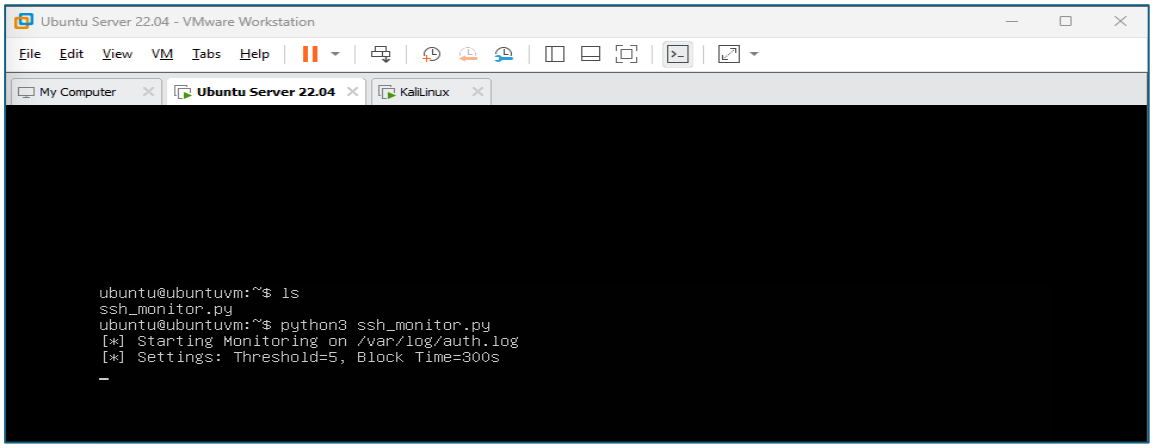
```
    for ip in list(blocked_ips.keys()):
        block_start = blocked_ips[ip]
        if current_time - block_start >= BLOCK_DURATION:
            print(f"[*] TIMEOUT: Unblocking {ip}...")
            try:
                subprocess.run(["sudo", "ufw", "delete", "deny", "from",
ip], check=True, capture_output=True)
                del blocked_ips[ip]
                # Reset failure count so they can be blocked again if they
retry
                failed_attempts[ip] = 0
            except subprocess.CalledProcessError as e:
                print(f"[ERROR] UFW unblock failed for {ip}: {e}")


def monitor_logs():
    """Main loop to tail the auth.log file."""
    print(f"[*] Starting Monitoring on {LOG_FILE}")
    print(f"[*] Settings: Threshold={THRESHOLD}, Block
Time={BLOCK_DURATION}s")


    try:
        # Open file and move pointer to the end (tail -f behavior)
        with open(LOG_FILE, "r") as f:
            f.seek(0, 2)

            while True:
                # CRITICAL FIX: Run unblock check every iteration,
                # even if no new log lines are coming in.
                unblock_ips()

                line = f.readline()
                if not line:
                    time.sleep(1) # Wait for new log entries
                    continue

                # Check for "Failed password" pattern
```

```
                if "Failed password" in line:
                    # Extract IP address using Regex
                    ip_match = re.search(r'from
(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})', line)
                    if ip_match:
                        ip = ip_match.group(1)
                        failed_attempts[ip] += 1
                        print(f"[ATTEMPT] {failed_attempts[ip]}/{THRESHOLD}
from {ip}")

                        if failed_attempts[ip] >= THRESHOLD and ip not in
blocked_ips:
                            block_ip(ip)

    except KeyboardInterrupt:
        print("\n[!] Monitor stopped by user.")
    except PermissionError:
        print("[ERROR] Access denied. Please run with 'sudo'.")
    except FileNotFoundError:
        print(f"[ERROR] Could not find {LOG_FILE}. Is SSH installed?")

if __name__ == "__main__":
    monitor_logs()
```

Save and exit.

Step 8: Run the Script

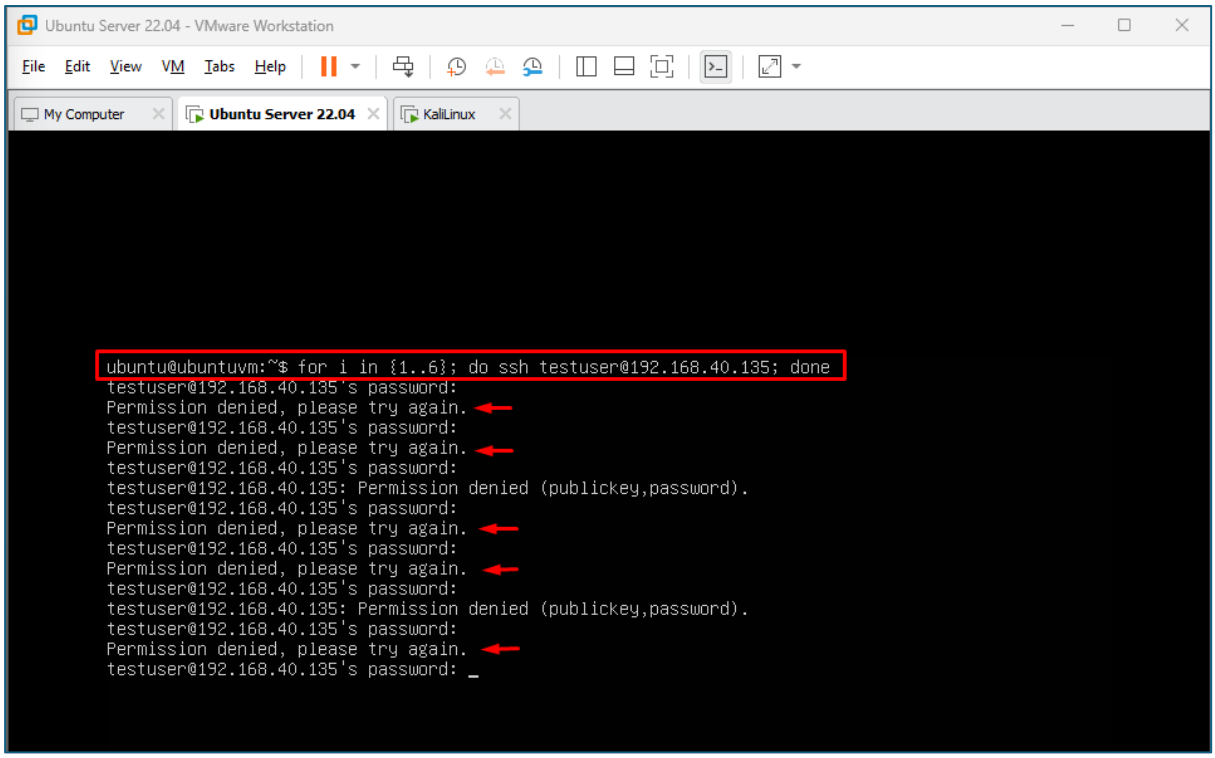
```
sudo python3 ssh_monitor.py
```



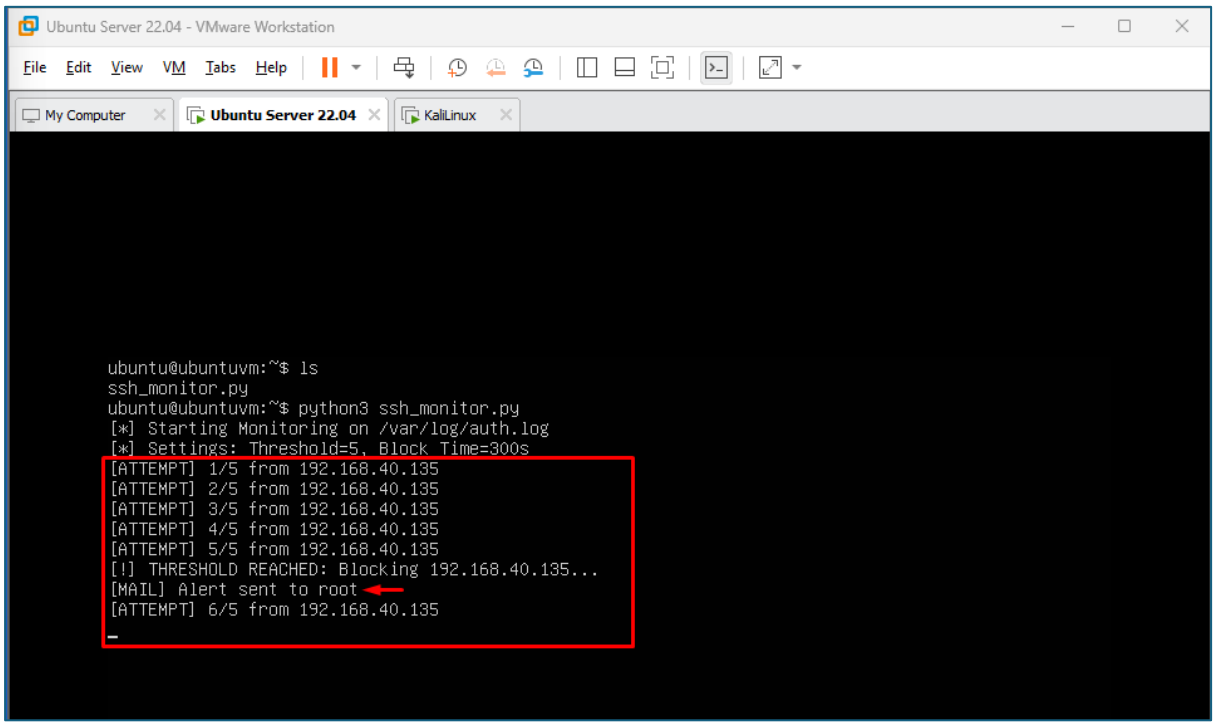
Step 9: Simulate Brute Force Attack

◆ From Same Machine

```
for i in {1..6}; do ssh testuser@192.168.40.135; done
```

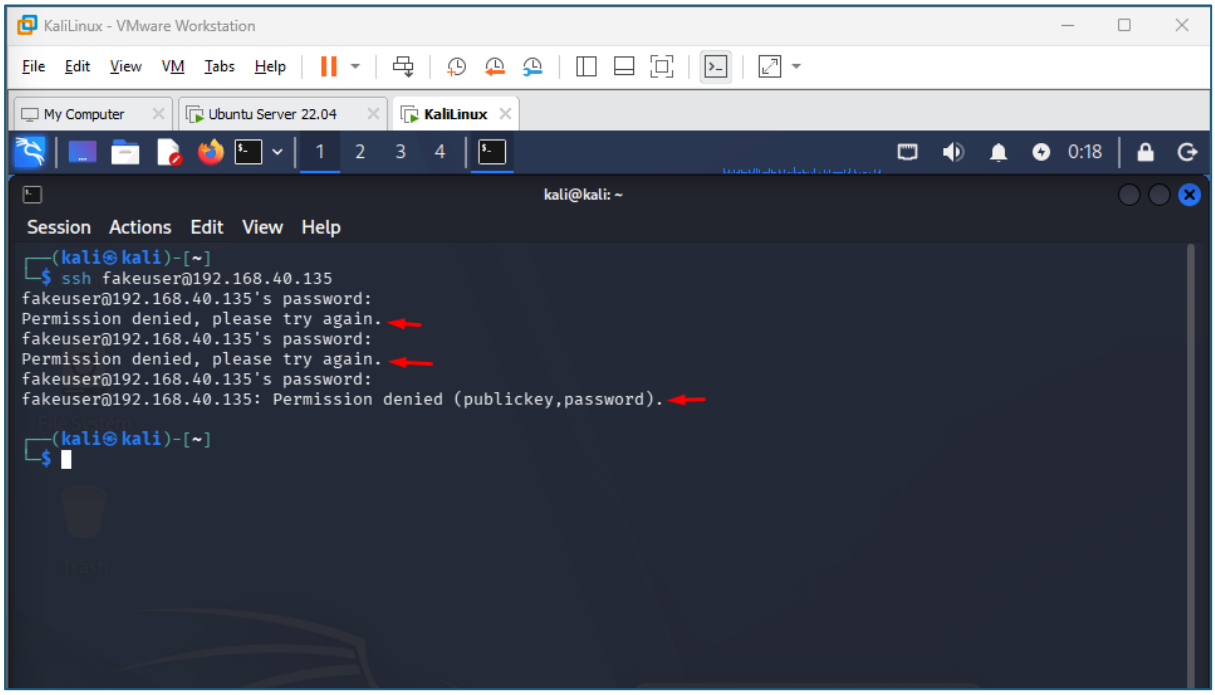


OUTPUT

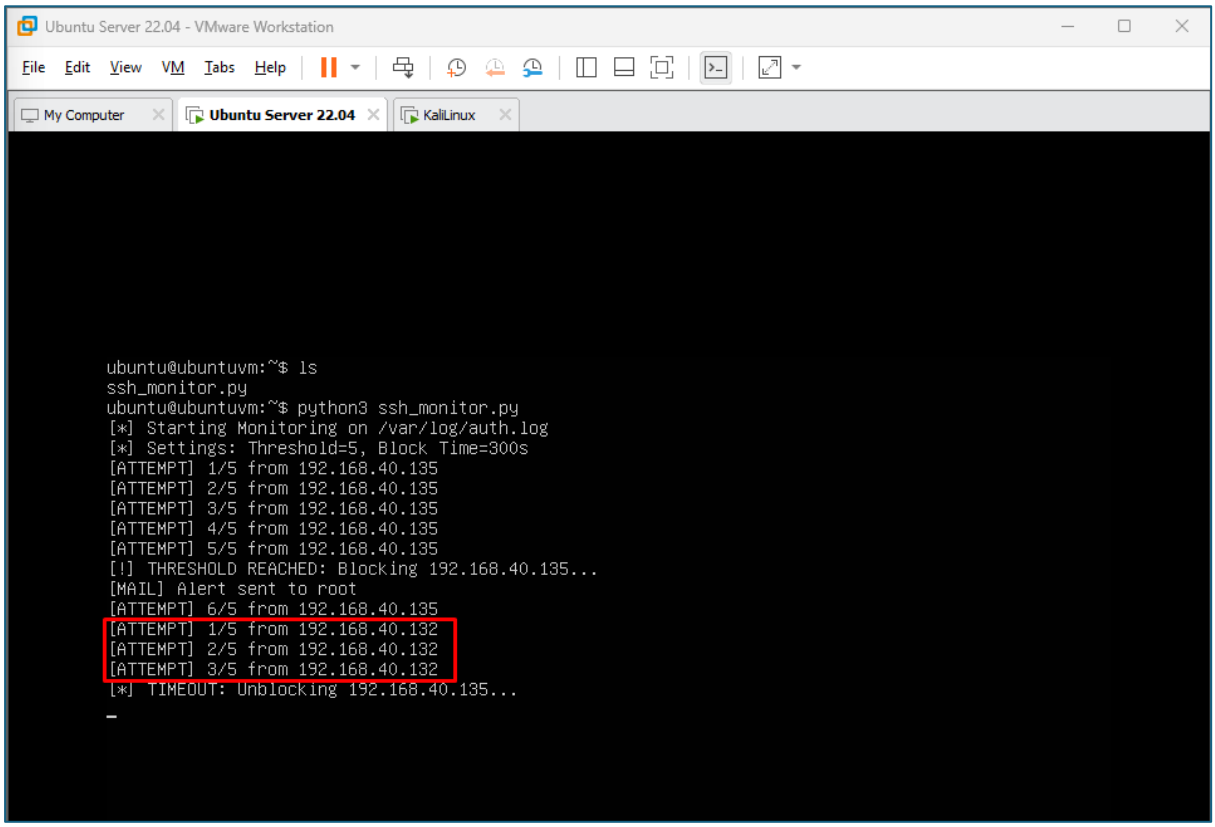


◆ From Another Machine

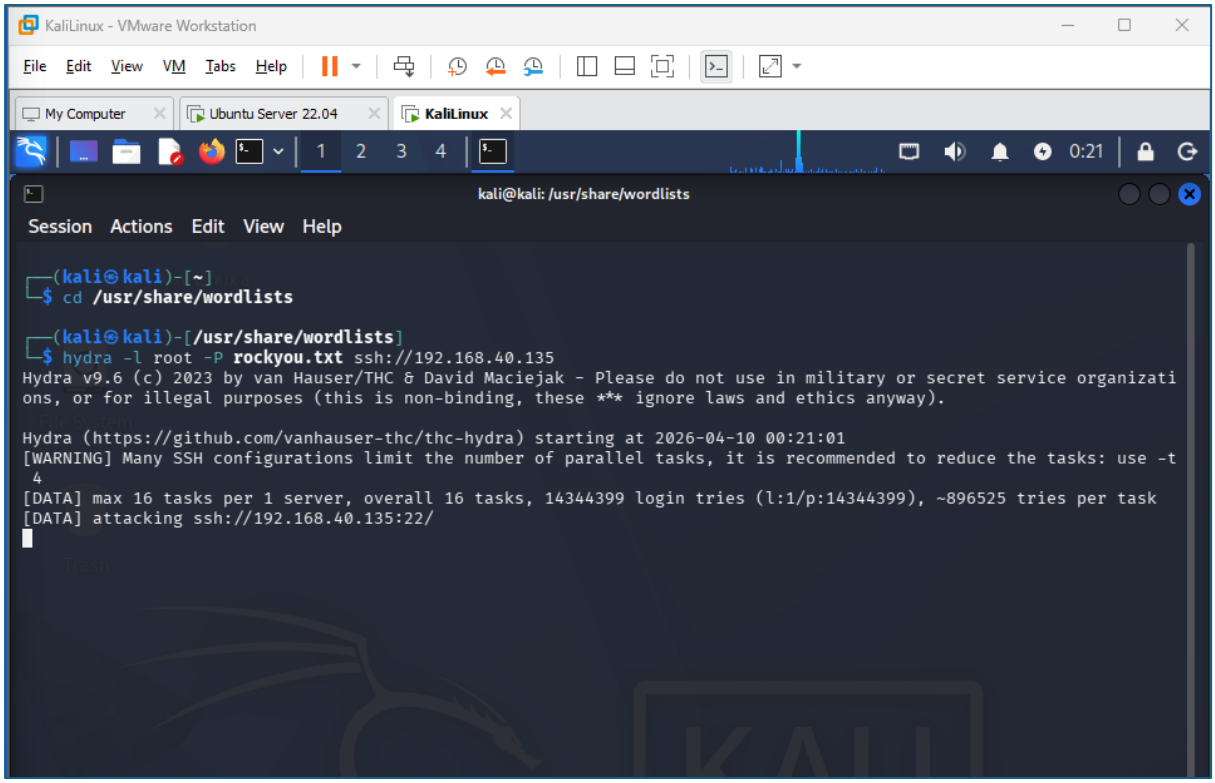
ssh fakeuser@192.168.40.135



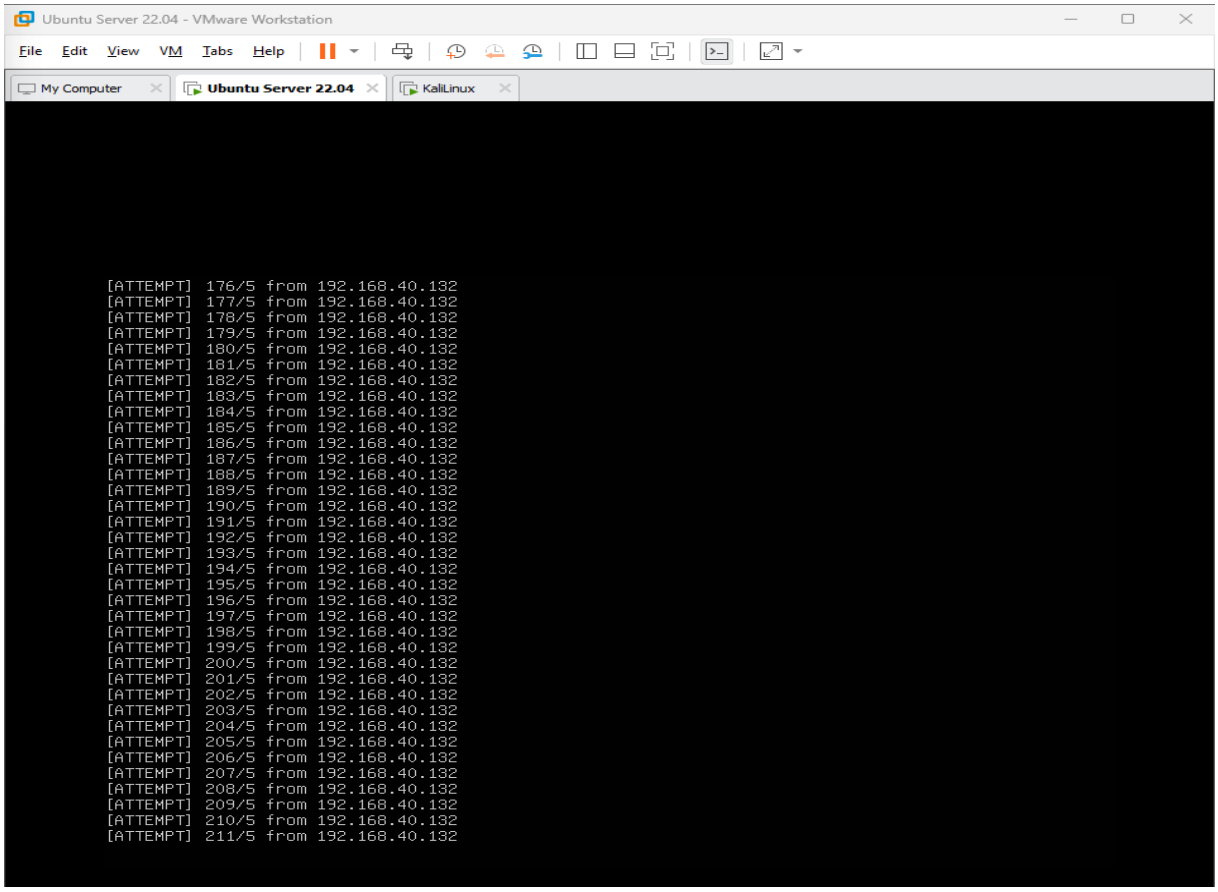
OUTPUT



```
hydra -l root -P rockyou.txt ssh://192.168.40.135
```



OUTPUT



Step 10: Observe Detection

Script output:

[!] Failed attempt from IP → Count: 339

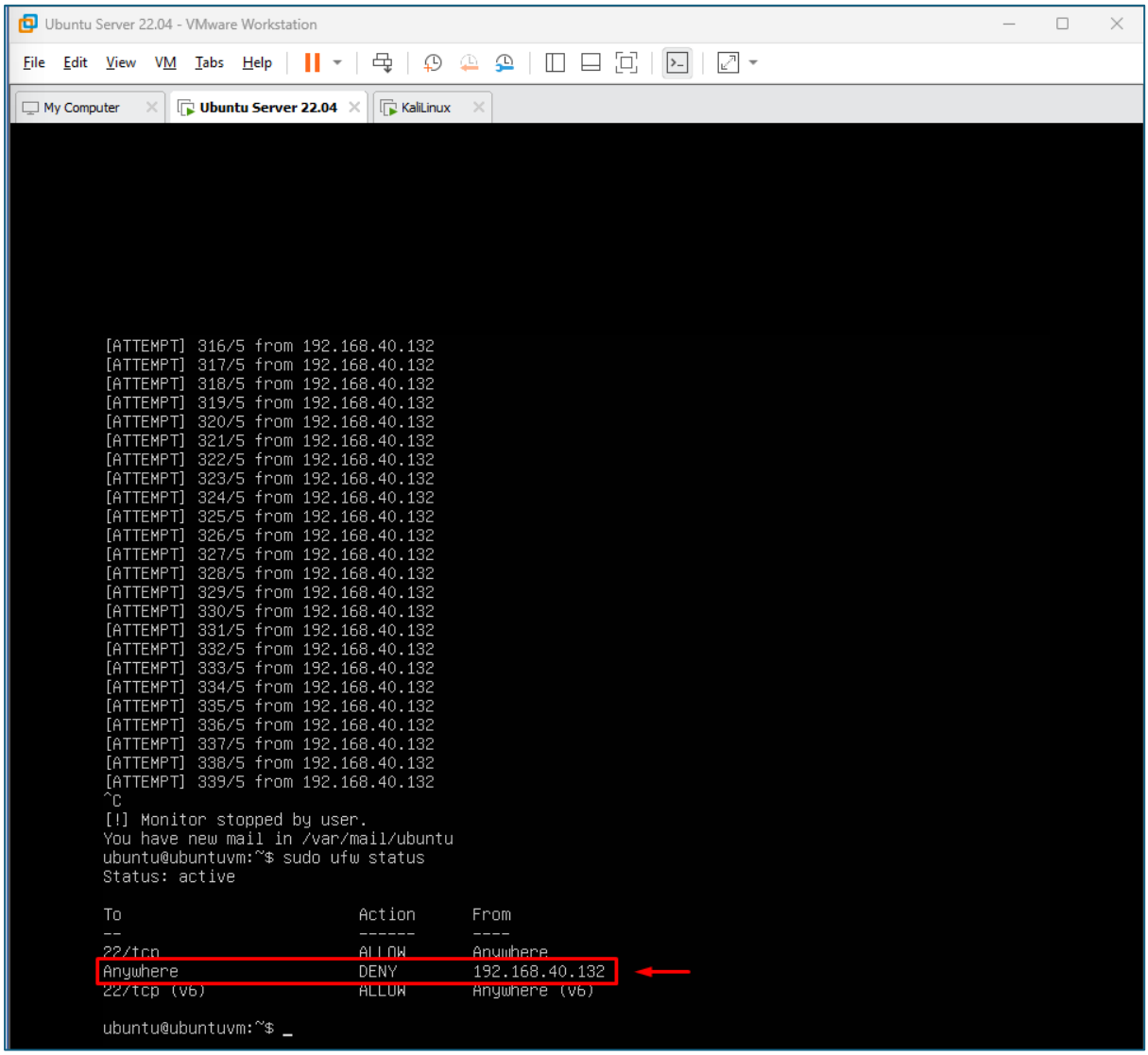
[+] Blocked IP: 192.168.40.132

Step 11: Verify IP Blocking

```
sudo ufw status
```

Expected output:

DENY 192.168.40.132



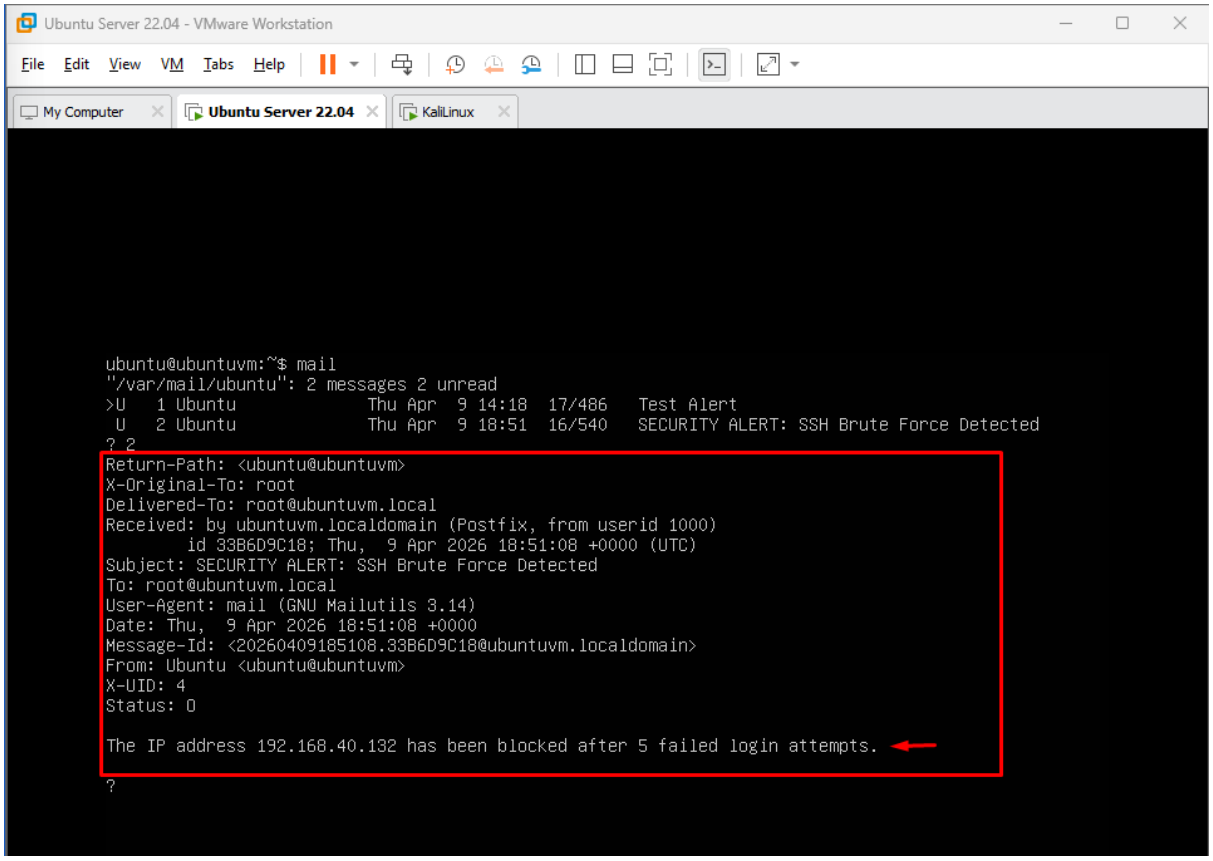
Step 12: Verify Alert Notification

```
mail
```

Expected:

Subject: SECURITY ALERT

Brute force attack detected from IP: 192.168.40.132



Step 13: Verify Auto-Unblocking

After configured time (~300 seconds):

```
sudo ufw status
```

Expected:

[-] Unblocked IP: 192.168.40.132

Expected Outcome

- Failed SSH attempts are detected in real time
- Attacker IP is blocked after threshold
- Alert is generated via Postfix mail
- IP is automatically unblocked after timeout

Key Learning Outcomes

- Log-based intrusion detection
- Firewall automation using UFW
- Python scripting for security automation
- Email alerting using local mail server
- Incident response workflow

Troubleshooting

Issue	Solution
Script not detecting logs	Run with sudo
Mail not received	Check postfix status
IP not blocked	Ensure UFW is enabled
Permission error	Use root privileges
No SSH logs	Verify SSH service running

Conclusion

This lab demonstrates a basic SOC automation workflow, combining:

- Detection (log monitoring)
- Response (IP blocking)
- Alerting (mail notification)

It significantly reduces manual monitoring effort and improves response time to brute-force attacks.